



Hawkins Experiments: Complete Specifications

Welcome to Hawkins Experiments! This comprehensive guide walks you through creating Code-Based (CB) tasks for model evaluation. You'll follow a two-branch workflow depending on whether your task involves **Refactoring/Maintainability (R/M)** or **Performance Optimization (PO)**.

📌 Your Mission: Provide all necessary artifacts (Prompt, Docker File, Scripts, and Golden Patch) for a task to be deterministically built and evaluated by our system.



Table of Contents

[Hawkins Experiments: Complete Specifications](#)

[Table of Contents](#)

[1. Project Overview](#)

[2. Key Deliverables](#)

[All Task Types](#)

[Refactoring/Maintainability Tasks](#)

[Performance Optimization Tasks](#)

[3. Workflow Overview](#)

[4. Common Steps \(1-4\)](#)

[Step 1: Understand the Source \(Provenance\)](#)

[Step 2: Configure the Docker File and provide task metadata](#)

[Step 3: Write the Prompt](#)

[Step 4 Set Task Type](#)

[5.1. Refactoring/Maintainability Path](#)

[Step 5 \(R/M\) Subtype Selection](#)

[Step 6 \(R/M\) Develop Run Script](#)

[5.2. Performance Optimization Path](#)

[Step 5 \(PO\) Category Selection](#)

[Step 6 \(PO\) Identify Target Functions](#)

[Step 7 \(PO\) Develop Reproduction Script](#)

[Step 8 \(R/M\) Develop Run Script](#)

[6. Final Steps \(9-14\)](#)

[Step 9: Add the parsing.py file](#)

[Step 10: Create Diff Patch/Golden Patch](#)

[Step 11: Run end-to-end checks \(e2e.sh\)](#)

[Step 12: Add before.json](#)

[Step 13: Add after.json](#)

[Step 14: Final Self-Check](#)

[Step 15 \(R/M\): Create Rubric](#)

[Step 16 \(R/M\): Grade Golden Patch With Rubric](#)

 [7. Final Submission Checklist](#)

 [8. Common Mistakes to Avoid](#)

 [9. FAQ & Troubleshooting](#)

 [10. Change Log](#)

1. Project Overview

The purpose of this project is to create a task structure that will evaluate an AI model's ability to make changes to a code repo that fix a specific problem **without introducing regressions** into the code base.

You will do this by creating a prompt that asks the model to carry out a specific action or solve a specific problem within a GitHub repo. Then, you will **create an environment** that can be used to **grade the model's response** to the prompt. You will not actually see the model's response in these tasks, but are just building the environment that can be used to grade it.

There are two different task types that you will be working with:

Refactoring/Maintainability (R/M) tasks and Performance Optimization (PO) tasks.

Depending on the task type that you select, the evaluation environment that you create will be slightly different.

Task Type	Description	Evaluation Method
-----------	-------------	-------------------

Refactoring/ Maintainability (R/M)	Improve code structure, migrate to new frameworks, update standards	Grading Rubric + Unit Tests
Performance Optimization (PO)	Improve execution speed, memory usage, or I/O efficiency	Quantitative Metrics + Unit Tests

The task type that you select will depend on the GitHub PR that you are assigned and on the prompt that you write.

2. Key Deliverables

These are the components that you will create for each task.

All Task Types

- **User-facing problem description**
- **Dockerfile**
- **Prompt**
- **Run Script**
- **Parsing file**
- **Golden Patch**

In addition to the components shown above, you will create the components shown below for the task type that you select.

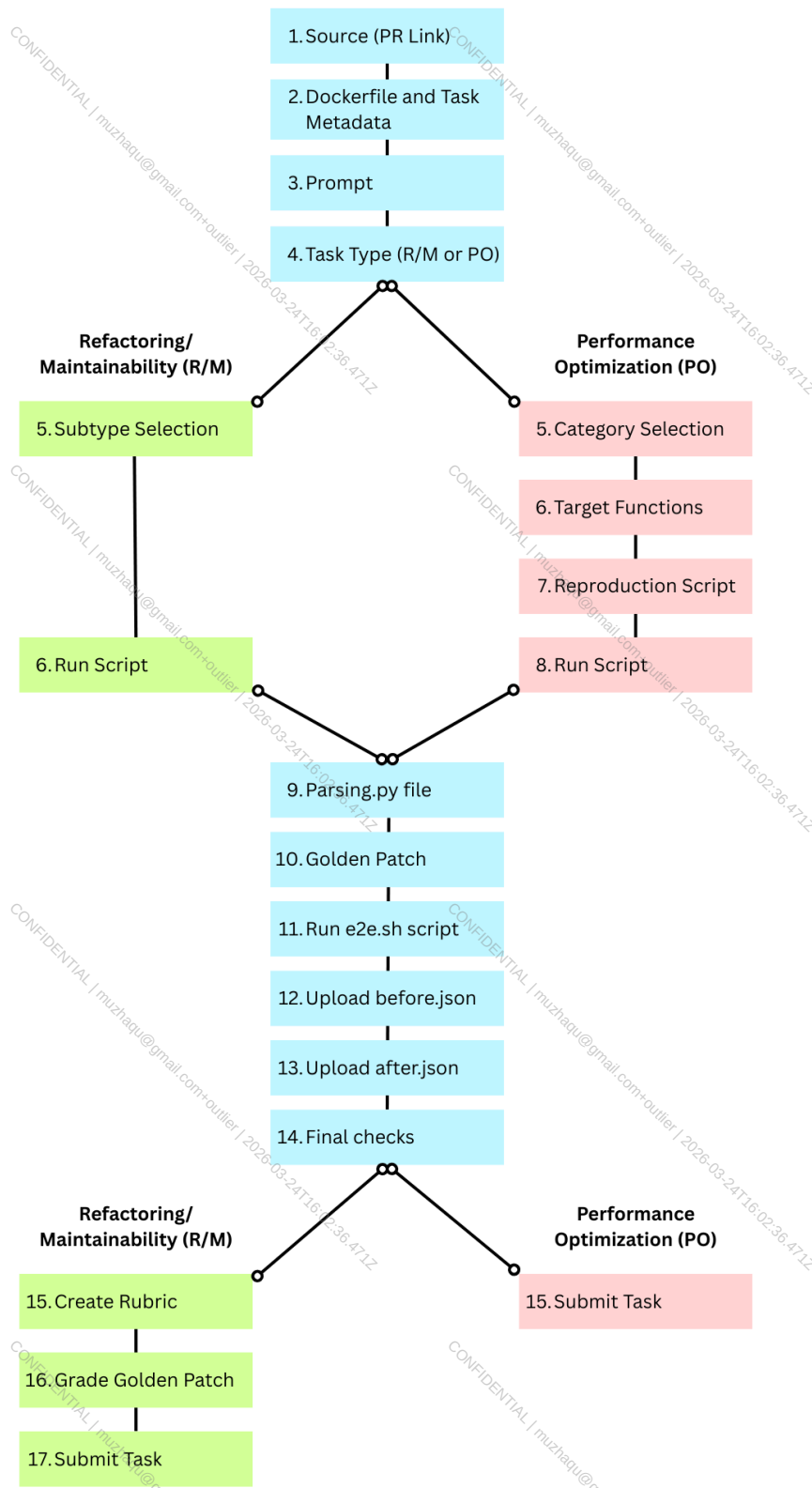
Refactoring/Maintainability (R/M) Tasks:

- **Grading Rubric**

Performance Optimization (PO) Tasks:

- Target Functions
- Reproduction Script

3. Workflow Overview



4. Common Steps (1-4)

These steps are required for **ALL** task types.

Step 1: Understand the Source (Provenance)

What: You will be provided with a GitHub link that will act as the source for your task. This link will lead to a GitHub PR.

You must open the provided link and understand the issue that is being solved by the PR. This issue is what you will use to write the user-facing problem description and the prompt for the task.

Step 2: Configure the Docker File and provide task metadata

1. Write a User-Facing Problem Description

- The user-facing problem description is a brief description of the problem that is presented in the provided GitHub link. Understand what issue the GitHub PR solves, and then create a brief summary or description of that issue as if it hasn't been solved yet.

2. You will create a Dockerfile that must:

- Clone/build the repo
- Install dependencies
- Checkout the parent commit of the PR
- Be reproducible

3. Identify the Primary Coding Language

- Identify the main programming language that will be used to solve the issue.

Dockerfile Template (MUST be called "Dockerfile")

FIND TEMPLATE HERE :

<https://app.outlier.ai/en/expert/guidelines/697aa99a5d1ac05a38db7e20?componentId=69b19a90f111b798070135cd&type=richText>

Dockerfile Guidelines

The Dockerfile that you create should follow the provided template. **ONLY** modify the sections that contain a `#TODO:` comment. Make sure that the Dockerfile:

- Checks out the **parent commit** of the Golden Patch. Do NOT check out the commit that includes the Golden Patch.

- Checks out the commit using a specific commit hash, not a branch name.
 - Installs ALL libraries and dependencies needed to run the reproduction_script.sh and run_script.sh files.
 - Does NOT specify or hard code a platform architecture (eg. x86_64, AArch64, etc.). It needs to be platform agnostic.
 - Does NOT run COPY commands for any of the run_script.sh, reproduction_script.sh, golden.patch, or parsing.py files.
-

Step 3 Write the Prompt

What: Create the prompt that the AI model will see. This prompt should describe the issue that the PR solves. In other words, write a prompt that will encourage the model to produce the patch that is committed in the PR, or something similar. Do not directly reference the patch or PR in the prompt, only describe the issue and request that it be fixed.

Prompt Requirements:

- **Self-contained:** All necessary information is contained in the prompt
- **Clear constraints:** What must/must not be changed
- **Defined "done":** How to know when the task is complete
- **Preserves Functionality:** Do not include a request in the prompt that changes the observable functionality of the code.
- **No solution:** Must NOT include the solution or direct hints

Prompt Structure Template:

```
## Context
[Brief background on the codebase and the problem]

## Problem Statement
[Clear description of what needs to be fixed/improved]

## Requirements
1. [Specific requirement 1]
2. [Specific requirement 2]
3. [Constraint - what NOT to change]

## Success Criteria
- [Measurable criterion 1]
- [Measurable criterion 2]
- [For PO tasks, include the performance that should be improved]
- [All existing tests must pass]

## Files to Modify (include all existing files that must be modified)
```

```
- `path/to/file1.py`  
- `path/to/file2.py`  
  
## Notes  
[Any additional context, but NO hints toward the solution]
```

! Critical Rule: The prompt must NEVER include:

- The solution code
- Specific algorithms to use
- Step-by-step implementation hints
- References to the PR, the run_script, the reproduction_script, the Golden Patch, the rubrics, etc.

✓ Example - Good Prompt

```
## Context  
The `DataProcessor` class in `src/processor.py` handles CSV file parsing  
for our data pipeline. Currently, it loads entire files into memory.  
  
## Problem Statement  
Large CSV files (>1GB) cause memory errors. Refactor the `process_file()`  
method to handle large files efficiently without loading everything into  
memory.  
  
## Requirements  
1. The `process_file()` method must handle files of any size  
2. Memory usage must not exceed 100MB regardless of file size  
3. All existing functionality must be preserved  
4. Do not modify the public API signature  
  
## Success Criteria  
- [ ] Memory usage stays under 100MB for 2GB test file  
- [ ] All existing unit tests pass  
- [ ] Processing time within 2x of current performance  
  
## Files to Modify  
- `src/processor.py`
```

Step 4 Set Task Type

What: Choose which workflow branch your task follows.

Value	Use When	Evaluation
-------	----------	------------

"Refactoring/ Maintainability (R/M)"	Code structure improvements, migrations, and standards updates	Grading Rubric + Unit Tests
"Performance Optimization (PO)"	Speed improvements, memory reduction, efficiency gains	Quantitative Metrics + Unit Tests

Each task will have a suggested task type:

- **The Task Type will be:** performance_optimization
- **The difficulty for this task will be:** medium
- **The authorized Github links are:**
<https://github.com/godotengine/godot/pull/94748>

While this suggestion is generally correct, you are responsible for the task type that is chosen. In the field shown below, you will choose the final task type for the task. You can choose the suggested task type if it is correct, or you can choose the other task type if the suggestion is incorrect. The task type that you select should be based off of the issue in the PR and the prompt that you write.

Task Type *

Select the type of task - this will determine which fields below are required. PT identify this task as [task_type placeholder], but please validate this decision

- ☐ Performance Optimization (PO)
- ☐ Refactoring/Maintainability (R/M)

5.1. Refactoring/Maintainability Path

Follow these steps only if task_type is "Refactoring/Maintainability."

Step 5 (R/M) Subtype Selection

What: Choose one subtype that best describes your refactoring task.

Subtype	Description	Example
Organization	Restructuring code for better maintainability	Breaking a monolithic file into modules
Memory Management	Fixing memory leaks or reducing the memory footprint	Fixing unclosed file handles
Latency	Reducing response times (non-algorithmic)	Adding caching to reduce API calls
Standards	Updating code to meet coding standards	Converting to PEP 8 compliance
Language	Migrating to a new language version	Python 2 to Python 3 migration

Framework	Upgrading or switching frameworks	Django 2.x to Django 4.x migration
-----------	-----------------------------------	------------------------------------

Step 6 (R/M) Develop Run Script (must be called “run_script.sh”)

What: The run script should run a set of unit tests that should be taken from the existing test suite within the GitHub repo you are working with. The run script should ONLY run the tests that already pass before the Golden Patch is applied.

The purpose of this script is to verify that the Golden Patch does not introduce regressions into the code. You will run this script before applying the Golden Patch to make sure that all tests pass. After that, you will apply the Golden Patch and then you will rerun this script to make sure that the same tests are all still passing.

Script Requirements:

- Run build/compile process (if applicable)
- Run linting checks (if applicable)
- Run existing passed unit tests

Template Script (run_script.sh)

Find template run script here:

<https://app.outlier.ai/en/expert/guidelines/697aa99a5d1ac05a38db7e20?componentId=69b19c0041e6fc11d2c27721&type=richText>

5.2. Performance Optimization Path

Follow these steps only if `task_type = "Performance Optimization"`

Step 5 (PO) Category Selection

What: Choose one category that best describes your optimization task.

Category	Description	Typical Metrics
Algorithmic	Improving time complexity	Execution time (ms), Operations count
Memory	Reducing memory usage	Peak memory (MB), Allocations
I/O	Optimizing file/network operations	Throughput (MB/s), Requests/sec
Vectorization	SIMD, GPU, or parallel optimizations	Speedup factor, FLOPS

Step 6 (PO) Identify Target Functions

What: List the specific function(s) that must be optimized. These are the existing functions in the code base that the model must identify as the ones needing modification in order to improve the performance of the code.

Requirements:

- Must be specific function names with file paths
- Functions must be the actual performance bottleneck
- Include class name if applicable

Example Format:

```
"src/data/processor.py::DataProcessor::process_batch",  
"src/utis/parser.py::parse_large_file"
```

Step 7 (PO) Develop Reproduction Script (must be called "reproduction_script.sh")

What: Write a script that measures the performance of the relevant code. The relevant code is the code that contains the performance bottleneck in the repo, and that must be optimized in the Golden Patch in order to improve performance. This script will test that the optimization provided in the Golden Patch has tangible, reproducible, and quantifiable improvements.

You will need to determine the best way to measure this performance. The performance metric could measure speed, memory, latency, operations per second, etc. The benchmark should evaluate a relevant performance metric that can be used to grade a response to your prompt. In other words, if a response to your prompt is high-quality, then the reproduction script should show a tangible, reproducible, and quantifiable improvement in the output performance metric when that response is applied to the repo.

NOTE: Your reproduction script **MUST** test the actual, relevant code that contains the performance bottleneck before the golden patch and is optimized after the golden patch. Creating simulated or dummy benchmarks that don't actually run the relevant code in the repo is strictly prohibited.

Script Requirements:

- Run build/compile process (if applicable)
- Create performance benchmark test
- Run performance benchmark test
- Output a single, parseable metric in this format: "METRIC_VALUE: <number>"
- Run deterministically

Example Script:

```
#!/bin/bash  
set -e  
  
cat << 'FILE_GODOT' > project.godot  
config_version=5  
[application]
```

```

config/name="TreePerfTest"

FILE_GODOT

cat << 'FILE_GD' > tree_benchmark.gd
extends SceneTree

func _init():
    var warmup_tree = Tree.new()
    var warmup_root = warmup_tree.create_item()
    for i in 20000:
        var item = warmup_tree.create_item(warmup_root)
    warmup_tree.free()

    var tree = Tree.new()
    var root = tree.create_item()
    root.set_text(0, "Root")

    var start_time = Time.get_ticks_msec()

    for i in 20000:
        var item = tree.create_item(root)
        item.set_text(0, "Item " + str(i))

    var end_time = Time.get_ticks_msec()

    print("METRIC_VALUE: ", end_time - start_time)

    tree.free()
    quit()
FILE_GD

BINARY=$(ls bin/godot.linuxbsd.editor.* 2>/dev/null | head -1)
scons platform=linuxbsd target=editor optimize=speed debug=symbols=no tests=yes
-j$(nproc)
BASELINE=$(($BINARY --headless -s tree_benchmark.gd 2>/dev/null | grep "METRIC_VALUE" |
awk '{print $2}')

echo -e "METRIC_VALUE: $BASELINE"

```

Remember, the reproduction script needs to measure the performance of the relevant code. You will run the script before applying the Golden Patch and capture the

performance metric. Then you will apply the Golden Patch. You will then rerun the reproduction script and capture the performance metric again. The metric from the first test without the patch will be compared to the metric from the second test with the patch to make sure that the Golden Patch is creating tangible, reproducible, and quantifiable improvements.

Step 8 (PO) Develop Run Script (must be called “run_script.sh”)

What: The run script should run a set of unit tests that should be taken from the existing test suite within the GitHub repo you are working with. The run script should ONLY run the tests that already pass before the Golden Patch is applied.

The purpose of this script is to verify that the Golden Patch does not introduce regressions into the code. You will run this script before applying the Golden Patch to make sure that all tests pass. After that, you will apply the Golden Patch and then you will rerun this script to make sure that the same tests are all still passing.

Script Requirements:

- Run build/compile process (if applicable)
- Run linting checks (if applicable)
- Run existing passed unit tests

Template Script (run_script.sh)

Find template run script here:

<https://app.outlier.ai/en/expert/guidelines/697aa99a5d1ac05a38db7e20?componentId=69b19c0041e6fc11d2c27721&type=richText>

6. Next Steps (9-14)

These steps are required for ALL task types.

Step 9: Add the parsing.py file (must be called “parsing.py”)

What: This is a file that takes in the results (stdout.txt and stderr.txt) of the run script (run_script.sh), parses them, and outputs a JSON file that follows the correct format. This happens automatically when you run e2e.sh (shown below). Use the parsing.py template that is given in the project resources. It implements most of the logic, but you

will need to modify the `parse\_test\_output` function. Only modify the `parse\_test\_output` function.

Template Script (parsing.py)

FIND HERE :

<https://app.outlier.ai/en/expert/guidelines/697aa99a5d1ac05a38db7e20?componentId=69b19ffcfd730684dd4241d&type=richText>

Step 10: Create Diff Patch/Golden Patch (must be called “golden.patch”)

What: Implement the "Gold Standard" solution as a diff patch. The Golden Patch should contain the changes made to the code repo in order to fully answer the prompt. The Golden Patch should be the perfect solution to ALL of the requests in the prompt.

The Golden Patch should only contain the changes necessary to answer the prompt, and should not include additional changes that go beyond the scope of the prompt.

The Golden Patch should follow best coding practices, with appropriate documentation (code comments, docstrings, etc), good naming conventions, and logical structures and formatting.

NOTE: You can only use the diff patch provided in the GitHub PR if it represents the perfect solution to your prompt. Otherwise, you must create the Golden Patch yourself.

Requirements:

- Must be a valid unified diff format
- Must have a .patch extension
- Must apply cleanly to the commit that is checked out in your Dockerfile
- Must pass ALL validation (e2e.sh) scripts
- Must fully solve the problem stated in the prompt
- Should follow best coding practices
- (R/M Tasks) Must pass ALL rubric criteria

Creating the Patch:

```
# Make your changes to the codebase  
# Then generate the patch:
```

```
git diff > golden.patch
```

```
# Or for staged changes:
```



```
git diff --cached > golden.patch
```

```
# Verify the patch applies cleanly:
```

```
git stash
```

```
git apply golden.patch
```

Patch Format Example:

```
diff --git a/src/processor.py b/src/processor.py
index abc1234..def5678 100644
--- a/src/processor.py
+++ b/src/processor.py
@@ -45,10 +45,15 @@ class DataProcessor:
     def process_file(self, filepath):
-        with open(filepath, 'r') as f:
-            data = f.read() # Loads entire file
-        return self._parse(data)
-        results = []
+        with open(filepath, 'r') as f:
+            for chunk in iter(lambda: f.read(8192), ''):
+                results.append(self._parse(chunk))
+        return self._merge_results(results)
```

Step 11: Run end-to-end checks on the task (using e2e.sh)

What: In order to ensure that all of the files in your task (Dockerfile, run_script.sh, reproduction_script.sh, golden.patch, and parsing.py) are working correctly, you must run the end-to-end task check using the e2e.sh script.

You can find the e2e.sh script here:

<https://app.outlier.ai/en/expert/guidelines/697aa99a5d1ac05a38db7e20?componentId=699e87e13ff45e697c58ca9d&type=richText>

Watch this video that shows you how to use and run the e2e.sh script:

<https://static.remotasks.com/uploads/65d91b8d9a8ee83f6c8da9d2/video1296351011.mp4>

NOTE: You CANNOT make any changes to the e2e.sh script. It must run fully without any errors and must produce the required output in order for you to submit your task. If the script does not run fully without errors or does not produce the correct output, then you must change YOUR files until it does.

The end-to-end script does the following:

- Builds a Docker image with your Dockerfile
- Runs the Docker container
- Runs the reproduction_script.sh file to measure pre-patch performance

- Runs the run_script file to make sure tests are passing
- Runs parsing.py with the output of the run_script
- Applies the golden patch
- Runs the reproduction_script.sh file to measure post-patch performance
- Runs the run_script file to make sure the same tests are still passing
- Runs parsing.py with the output of the run_script
- Creates several new files with output

Explanation of the files that are created by the end-to-end script:

- **"before.json"**: This file contains the formatted tests that are run by the run_script before the patch is applied. All tests in this file must show a status of "PASSED". You will upload this file to the task.
- **"after.json"**: This file contains the formatted tests that are run by the run_script after the patch is applied. All tests in this file must show a status of "PASSED". All tests present in "before.json" must be present in this file. You will upload this file to the task.
- **"fail_to_fail.json"**: This file contains any tests that are run by the run_script that fail before and after the patch is applied. This file should contain an empty array. If this file contains a non-empty array, then you need to make changes to your run_script or to the golden patch and re-run the e2e script until this shows an empty array.
- **"fail_to_pass.json"**: This file contains any tests that are run by the run_script that fail before the patch is applied, but pass after the patch is applied. This file should contain an empty array. If this file contains a non-empty array, then you need to make changes to your run_script or to the golden patch and re-run the e2e script until this shows an empty array.
- **"pass_to_pass.json"**: This file contains an array of tests that are run by the run_script that pass before and after the patch is applied. This array should contain all of the tests that appear in "before.json" and "after.json".
- **"golden_patch_apply_stderr.txt"**: This file will contain any error messages that may occur if your Golden Patch fails to apply cleanly. This file should be blank. If this file is not blank, you will need to troubleshoot your Dockerfile and Golden Patch to figure out why the patch is not applying.
- **"reproduction_before_stdout.txt/reproduction_before_stderr.txt"**: These files contain the output of running your reproduction script before the golden patch is applied. The stdout.txt file should contain the performance metric: "METRIC_VALUE: <number>". You will need to copy and paste this output into the task. The stderr.txt file will contain any error messages that occurred when running the reproduction script.
- **"reproduction_after_stdout.txt/reproduction_after_stderr.txt"**: These files contain the output of running your reproduction script after the golden patch is applied. The stdout.txt file should contain the performance metric: "METRIC_VALUE: <number>". You will need to copy and paste this output into the

task. The stderr.txt file will contain any error messages that occurred when running the reproduction script.

- **"run_before_stdout.txt/run_before_stderr.txt":** These files contain the output of running your run_script before the golden patch is applied. The stdout.txt file should contain any output from the tests running successfully. The stderr.txt file should contain any error messages that occurred when running the run_script. If there are error messages, you may need to troubleshoot to make sure that all required tests are running properly.
- **"run_after_stdout.txt/run_after_stderr.txt":** These files contain the output of running your run_script before the golden patch is applied. The stdout.txt file should contain any output from the tests running successfully. The stderr.txt file should contain any error messages that occurred when running the run_script. If there are error messages, you may need to troubleshoot to make sure that all required tests are running properly.

Step 12: Upload the "before.json" file that is produced after you run e2e.sh.

What: This file is automatically created when you run the e2e.sh script. Make sure that this file is not empty, and that it only contains tests with a "status" of "PASSED". Make sure that all passing tests in this file match a passing test in `after.json`. If there is a mismatch in tests between `before.json` and `after.json`, you must change your `run\_script` and rerun e2e.sh so that they match. Verify that the file contains json that follows the format shown below. Upload this file after running the e2e.sh script.

```
"tests": [  
  {  
    "name": "tests/middleware/test_gzip.py::test_gzip_responses[asyncio]",  
    "status": "PASSED"  
  },  
  {  
    "name":  
"tests/middleware/test_gzip.py::test_gzip_not_in_accept_encoding[asyncio]",  
    "status": "PASSED"  
  }  
]
```

Step 13: Upload the "after.json" file that is produced after you run e2e.sh.

What: This file is automatically created when you run the e2e.sh script. Make sure that this file is not empty, and that it only contains tests with a "status" of "PASSED". Make

sure that all passing tests in this file match a passing test in `before.json`. If there is a mismatch in tests between `before.json` and `after.json`, you must change your `run\_script` and rerun e2e.sh so that they match. Verify that the file contains json that follows the format shown below. Upload this file after running the e2e.sh script.

```
"tests": [
  {
    "name": "tests/middleware/test_gzip.py::test_gzip_responses[asyncio]",
    "status": "PASSED"
  },
  {
    "name":
"tests/middleware/test_gzip.py::test_gzip_not_in_accept_encoding[asyncio]",
    "status": "PASSED"
  }
]
```

NOTE: Make sure that “before.json” and “after.json” are matching after running the e2e.sh script. The exact same tests should be present in both files with a status of PASSED. If either file contains extra tests, is missing tests, or contains a test that has a status other than PASSED, then you must change your run_script file or golden patch so that all tests pass before the patch is applied and are still passing after the patch is applied. Exclude any tests that are not already passing before the patch is applied.

Step 14: Final Self-Check

What: Verify everything works together before submission. You MUST run e2e.sh with your files to verify that everything is working. Do NOT submit your task if e2e.sh does not work or does not produce the correct output.

Self-Check Checklist:

- Docker image builds successfully
- e2e.sh script runs without errors
- Gold patch applies cleanly
- Golden patch fully answers the prompt
- All tests pass before the gold patch is applied
- All tests pass after the gold patch is applied

- before.json and after.json are identical
- Reproduction script runs before the gold patch is applied
- Reproduction script runs after the gold patch is applied
- Reproduction script shows tangible, reproducible, and quantifiable improvements.
- Prompt is clear and complete
- Prompt does NOT contain solution hints
- All required fields are filled
- Source URL is valid and accessible
- Artifacts are aligned (prompt matches solution matches tests)

Step 15 (R/M): Write a Rubric with Success Criteria

What: Write clear rubrics for the LLM judge to evaluate the solution ([Rubric Guide down below](#)). The rubric should be written based on the prompt that you created. The rubric should be a comprehensive, self-contained evaluation of how well a response fully fulfilled all prompt requests.

There will be an LLM rubric generator to help build the rubric criteria. Those LLM-generated rubrics MUST be revised thoroughly and edited to fit the guidelines and requirements.

WARNING: The rubric-generating model can make mistakes that must not be overlooked. These LLM-generated rubrics should not be treated as a ground truth, but as a starting point and inspiration for the complete set that you must create. You will need to modify, add, and delete criteria from the LLM-generated rubric in order to create the perfect rubric that you will submit with the task.

DISCLAIMER: Leaving the rubrics as is, without fine-tuning them, will result in being disabled from the project.



Expert Rubric

A **rubric** is a set of clear, specific requirements that describe what a strong response to your prompt must contain. You must create a rubric document tailored specifically to your prompt's requirements with a **minimum of 15 criteria and a maximum 20 criteria**.

Rubric Requirements:

- Must comprehensively cover ALL explicit and implicit requests in the prompt that aren't already checked in the run_script, reproduction_script, or e2e.sh. If the rubric fails to evaluate something that is requested in the prompt and is not checked by the run_script or reproduction_script, your task will fail.
- Must accurately evaluate ALL possible correct solutions to the prompt, not only your specific Golden Patch.

Criteria Requirements:

- **Rubric-Specific:** Rubric criteria must only check parts of the solution that are not already verified in the `run_script` or `reproduction_script`. For example, you must NOT include any rubric criteria that talk about unit tests passing, since that is already verified in the `run_script`.
- **Atomic:** Each criterion checks only one idea. A criterion should not evaluate multiple, unrelated things.
- **Objective:** Each criterion must objectively evaluate a specific part of a response. When grading a response, you should be able to "PASS" or "FAIL" each criterion objectively, without any subjective ambiguity.
- **Correctly Framed:** Each criterion should be written in such a way that a good response would evaluate to "Yes" or "True" in order for that criterion to "PASS".
- **Positive:** Uses positive phrasing (e.g., "Code includes..." vs "Code doesn't forget...").
- **Accurate:** Each criterion should accurately evaluate part of the prompt. For example, if the prompt says, "Include 3 new functions that do...", then you should NOT have a criterion that says, "The solution includes 4 new functions that do...".
- **Scoped:** Each criterion should evaluate something that is explicitly or implicitly requested in the prompt. Do not include any criteria that check for something that is not requested in the prompt.
- **Self-Contained:** Each criterion needs to be able to be evaluated ONLY within the context of the criterion itself and the response diff patch. In order to accurately grade the criterion, you should not need to reference the prompt, the PR, any of the scripts, or any other outside material. Include all information that is needed to evaluate the criterion within the criterion itself. (Bad Example: "The solution correctly implements the sorting logic on the input array." vs. Good Example: "The solution implements merge sorting logic on the array that is received as input in the `test_name`` function.")
- **Unique:** Each criterion should evaluate something unique. There should be no redundancy or overlapping between multiple criteria.
- **Not Overfitting or Underfitting:** Criteria should be able to accurately grade ALL correct solutions based on the requirements and requests in the prompt. Avoid writing criteria that are overly specific to the point that they would fail a good response that accurately follows the prompt. On the flip side, avoid writing criteria that are so generic that they pass a bad response that does not follow the prompt.

Rubric Weights

Each rubric criterion must have **one** of the following weights:

- **5 — Mandatory**

This is a core requirement for adequately answering the prompt, is clearly expected based on the prompt, and it would be hard to envision an acceptable response without it.

- **3 — Important**

This makes the response substantially better, and is reasonably or implicitly expected based on the prompt, though you can envision an acceptable response without it as long as all other criteria are met.

- **1 — Nice to have (Optional)**

This is a nice-to-have and necessary for a perfect response, but a response can still be strong without it.

⚠ **Do NOT select 2 or 4 as weights. The only allowed options are 1, 3, or 5.** ⚠

Formatting:

Rubric criteria should follow these formatting guidelines:

- Should NOT be written in past tense.
- Should NOT be written as a question.
- Should NOT begin with: "The model..."

Scoring Rules

For each rubric item, you should be able to determine if a given response is a "PASS" or a "FAIL".

- **Criteria Fully Met (PASS)**
- **Not Met (FAIL)**

NOTE: Your Golden Patch MUST have all criteria as Pass

Criteria Categories

1. **Instruction Following**

Ensures the response adheres to explicit directions in the prompt (format, constraints, language, libraries, required elements).

2. **Code Correctness**

Ensures any code performs the intended task and produces correct results based on the prompt.

3. **Code Clarity**

Covers readable, well-structured code, including naming, organization, and formatting.

4. **Efficiency**

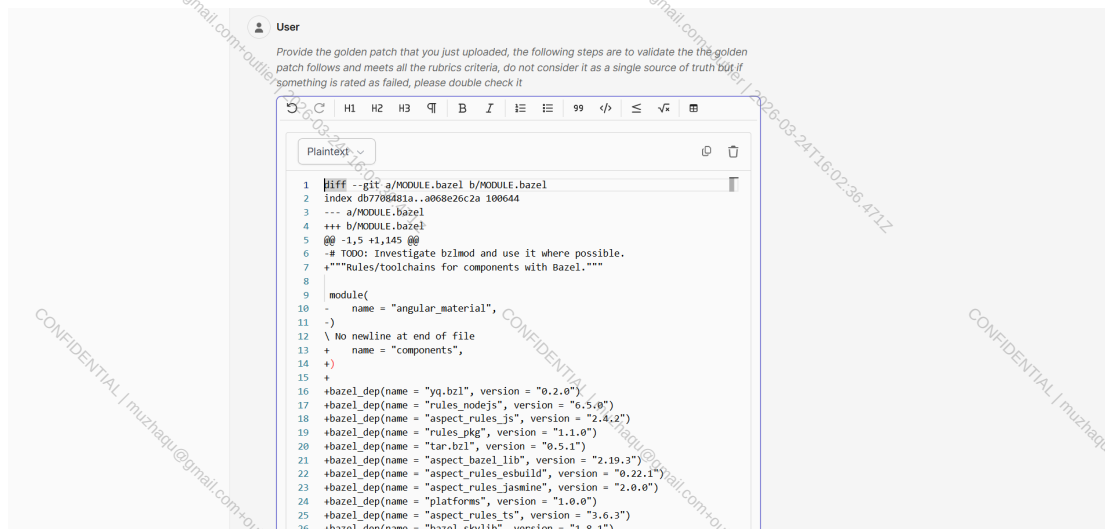
Covers conciseness, avoidance of unnecessary steps, and reduction of redundancy.

5. **Risk Security**

Ensures a safe response that avoids inappropriate or harmful content.

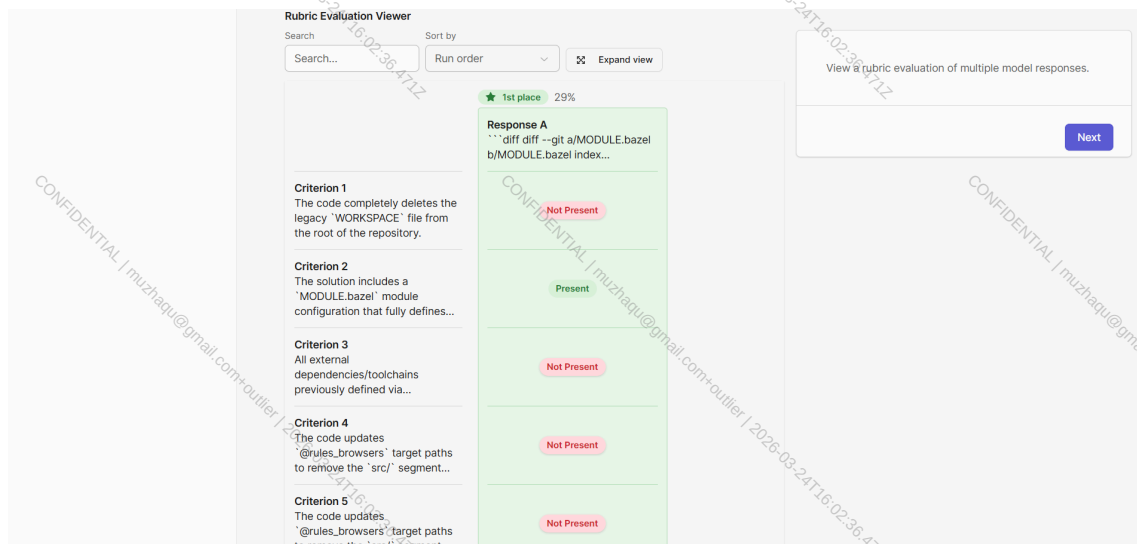
Step 16 (R/M): Grade Golden Patch With Rubric

What: Validate that the golden patch actually covers and addresses all rubric criteria properly. The code from your “golden.patch” file must be pasted in this section to be evaluated against the rubric set that was previously created. Each rubric will be scored with either a 1 (pass, meaning the patch covers the rubric) or a 0 (fail, meaning the patch doesn’t address that rubric).



The screenshot shows a code editor with a file named "golden.patch". The patch content is as follows:

```
diff --git a/MODULE.bazel b/MODULE.bazel
index db7708481a..a068e26c2a 100644
--- a/MODULE.bazel
+++ b/MODULE.bazel
@@ -1,5 +1,145 @@
-# TODO: Investigate bazmod and use it where possible.
+""Rules/toolchains for components with Bazel.""
+
+module(
+  name = "angular_material",
+  >
+  \ No newline at end of file
+  + name = "components",
+  +
+  +bazel_dep(name = "yq.bzl", version = "0.2.0")
+  +bazel_dep(name = "rules_nodejs", version = "6.5.0")
+  +bazel_dep(name = "aspect_rules_js", version = "2.4.2")
+  +bazel_dep(name = "rules_pkg", version = "1.1.0")
+  +bazel_dep(name = "tar.bzl", version = "0.5.1")
+  +bazel_dep(name = "aspect_bazel_lib", version = "2.19.3")
+  +bazel_dep(name = "aspect_rules_esbuild", version = "0.22.1")
+  +bazel_dep(name = "aspect_rules_jasmine", version = "2.0.0")
+  +bazel_dep(name = "platforms", version = "1.0.0")
+  +bazel_dep(name = "aspect_rules_ts", version = "3.6.3")
+  +bazel_dep(name = "bazel_skylib", version = "1.8.1")
```



The screenshot shows the "Rubric Evaluation Viewer" interface. It displays a table with five criteria and their evaluation results. The table is sorted by "Run order".

Criterion	Response A
Criterion 1 The code completely deletes the legacy 'WORKSPACE' file from the root of the repository.	Not Present
Criterion 2 The solution includes a 'MODULE.bazel' module configuration that fully defines...	Present
Criterion 3 All external dependencies/toolchains previously defined via...	Not Present
Criterion 4 The code updates '@rules_browsers' target paths to remove the 'src' segment...	Not Present
Criterion 5 The code updates '@rules_browsers' target paths to remove the 'src' segment...	Not Present

On the right side of the interface, there is a "Next" button and a message: "View a rubric evaluation of multiple model responses."

If there's any rubric marked with a 0 (fail), there are two paths that can be taken:

1. If the Golden patch doesn't cover the failed criterion, it has to be updated to address it completely. Then run the eval once more to make sure it passes.

Collect text inputs from the user

3/3 completed

Does the patch pass all the criteria?

If you select no, please validate if the model is overflagging or iterate on the rubrics to fix it

☐ Yes

☒ No

Did you agree with the rubrics ratings from the top? *

☒ Yes

☐ No

Please iterate on the criteria to fix it

Please mark this field after you apply the changes

☒ Fixed

Next

2. If it is considered that the golden patch does address that specific criterion, despite what the eval says, then it will be necessary to write a detailed justification as to where exactly in the golden patch that rubric is being addressed, and why that specific part of the patch addresses that rubric. For this specific scenario, the section asking "Does the patch pass all the criteria?" should be answered as "No". And in the following pop-up asking, "Did you agree with the rubric's ratings from the top?", the answer should be "No" as well. Then the detailed justification should be written in the text box.

Collect text inputs from the user

2/3 completed

Does the patch pass all the criteria?

If you select no, please validate if the model is overflagging or iterate on the rubrics to fix it

☐ Yes

☒ No

Did you agree with the rubrics ratings from the top? *

☐ Yes

☒ No

Write your justifications on why you disagree *

Next

✓ 7. Final Submission Checklist

Before Submitting, Verify:

Source & Prompt

- Source URL is valid and publicly accessible
- Prompt aligns with source issue/PR
- Prompt is self-contained (no external references needed)
- Prompt does NOT contain solution or hints
- Success criteria are clear and measurable

Docker & Environment (Dockerfile)

- Dockerfile builds without errors when run with `e2e.sh`

- Uses specific commit hash (not branch name)
- Commit hash is the parent of the golden patch commit (the golden patch needs to apply cleanly to this commit hash)
- All dependencies are installed
- Environment is deterministic
- Environment is platform-agnostic

Reproduction Script (reproduction_script.sh)

- File is named "reproduction_script.sh"
- Script runs successfully when run with e2e.sh
- Script creates a benchmark test that evaluates the performance of code that is relevant to the prompt.
- The benchmark test actually runs relevant code from the repo, not simulated code
- Script contains a rebuild/recompile step when necessary
- Script runs the benchmark test
- Script outputs a single, parseable metric in the format of: "METRIC_VALUE: <number>"

Run Script (run_script.sh)

- File is named "run_script.sh"
- Run script runs successfully when run with e2e.sh
- Script contains a rebuild/recompile step when necessary
- Run script runs unit tests that pass before and after the golden patch is applied

Golden Patch (golden.patch)

- Patch applies cleanly to the commit that is checked out in the Dockerfile
- Solution passes all end-to-end checks with e2e.sh
- Solution fully addresses the prompt
- Solution represents best practices
- Solution passes all rubric criteria

Path-Specific (R/M)

- Subtype is correctly selected
- Success criteria rubric is complete
- LLM judge criteria are objective

Path-Specific (PO)

- Category is correctly selected
- Target functions are specific
- Metric unit is defined
- Unit test command is specified
- Benchmark is deterministic

End-to-End Test (e2e.sh)

- You make NO changes to the e2e.sh file (run the script exactly as it is)
- The script runs fully without errors
- The script produces accurate before.json and after.json files
- The script produces accurate reproduction script outputs (found in reproduction_before_stdout.txt and reproduction_after_stdout.txt)

8. Common Mistakes to Avoid

Mistake	Why It's Wrong	How to Fix
Including solution hints in the prompt	Makes the task too easy; doesn't test real problem-solving	Have someone else review the prompt blind
Using branch names instead of commit hashes	Breaks reproducibility when branch changes	Always use specific commit SHA

Non-deterministic benchmarks	Results vary between runs, invalidating metrics	Use fixed seeds, multiple iterations, warm-up runs
Vague success criteria	LLM judge cannot evaluate consistently	Make all criteria binary (pass/fail) or numeric
Missing edge cases in tests	Gold solution might not actually be correct	Test with edge cases, empty inputs, large inputs
Overly broad scope	Task becomes too complex to evaluate	Focus on specific, well-defined problems
Golden Patch doesn't apply	This prevents the run_script.sh and reproduction_script.sh files from running properly	Make sure that your Dockerfile checks out the parent commit to the Golden Patch
End-to-End script fails	Task files (Dockerfile, run_script.sh, reproduction_script.sh, golden.patch, parsing.py) have an error	Review the output of the e2e.sh run in the files produced (*_stderr.txt files) to find the issue.

? 9. FAQ & Troubleshooting

Troubleshooting Docker build fails:

- Check that the repository URL is correct
- Verify the commit hash exists
- Ensure all system dependencies are listed

Patch doesn't apply:

- Ensure you're on the correct commit
- Check for whitespace issues in the patch
- Regenerate the patch with `git diff --no-color`

Non-deterministic metrics:

- Add warm-up iterations before measurement
- Use fixed random seeds
- Take median of multiple runs, not single measurement
- Disable CPU frequency scaling if possible



10. Change Log

Version	Date	Changes
5.0	Mar 10, 2026	Updates prompt, golden patch, rubric sections. Includes more details throughout. Removes outdated information.
4.0	Feb 24, 2026	Based on the new taxonomy, new file names, e2e.sh requirement
3.0	Feb 16, 2026	Based on the new taxonomy, the required fields.
2.0	2026-01-29	Complete rewrite with detailed examples, templates, and a troubleshooting guide
1.0	Initial	Original specifications document

Questions? Contact the Hawkins Experiments team.